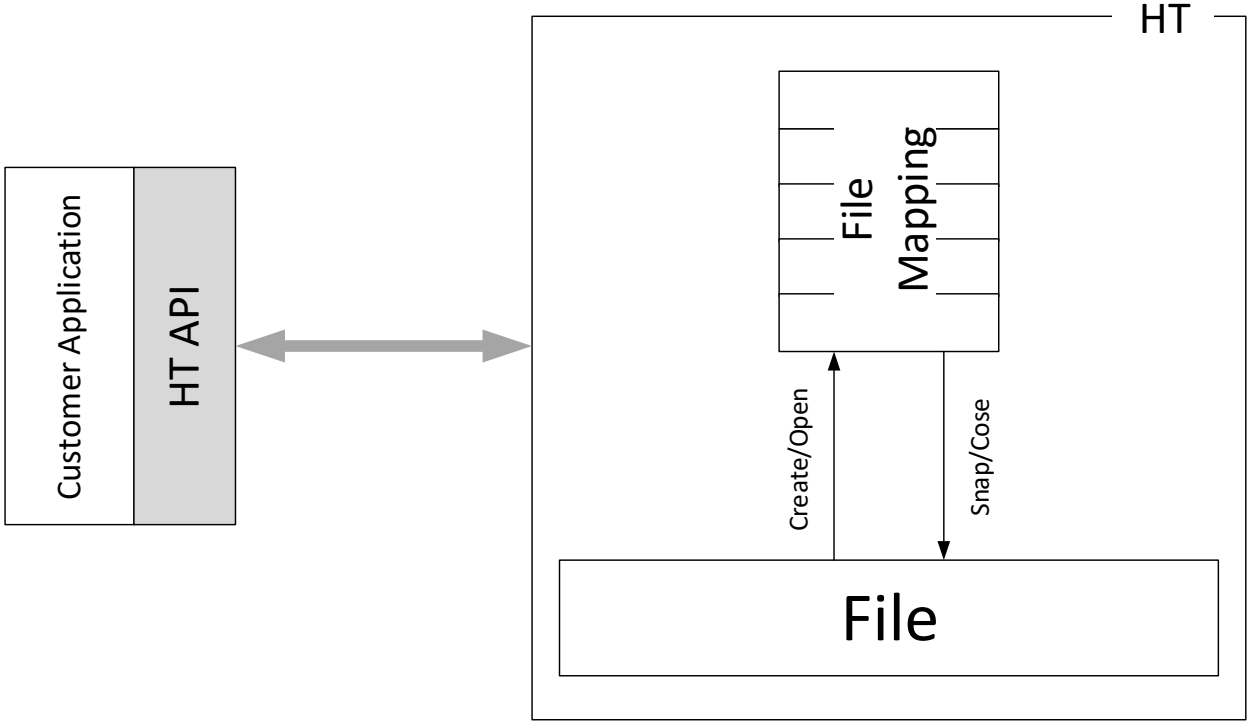
Лабораторная работа 10
File mapping
ПОИТ-3, ПОИБМС
6 часов

1. ***Внимание! Для работы с файловой системой использовать только OS API.***
2. **HT-хранилище (HT):** программная система, предназначенная для хранения данных в формате ключ/значение. Для хранения данных в HT используется файл на диске и *образ* этого файла в оперативной памяти. Операции с данными (Insert, Delete, Update, Get) выполняются с *образом*. Данные образа организованы в виде хэш-таблицы с открытой адресацией. Сохранность данных осуществляется с помощью асинхронного *snapshot*- механизма (с заданным периодом асинхронное сохранение образа в файле), *snapshot* может выполняться синхронно по команде пользователя. Для использования HT в приложении предназначено потокобезопасное HT API.



3. Спецификация HT API

```
namespace HT    // HT API
{
    // API HT - программный интерфейс для доступа к HT-хранилищу
    //          HT-хранилище предназначено для хранения данных в ОП в формате ключ/значение
    //          Персистестеность (сохранность) данных обеспечивается с помощью snapshot-менханизма
    //          Create - создать  и открыть HT-хранилище для использования
    //          Open   - открыть HT-хранилище для использования
    //          Insert - создать элемент данных
    //          Delete - удалить элемент данных
    //          Get    - читать  элемент данных
    //          Update - изменить элемент данных
    //          Snap   - выпонить snapshot
    //          Close  - выполнить Snap и закрыть HT-хранилище для использования
    //          GetLastError - получить сообщение о последней ошибке
```

```
struct HTHANDLE    // блок управления HT
{
    HTHANDLE();
    HTHANDLE(int Capacity, int SecSnapshotInterval, int MaxKeyLength, int MaxPayloadLength, const char FileName[512]);
    int     Capacity;               // емкость хранилища в количестве элементов
    int     SecSnapshotInterval;    // периодичность сохранения в сек.
    int     MaxKeyLength;           // максимальная длина ключа
    int     MaxPayloadLength;       // максимальная длина данных
    char    FileName[512];          // имя файла
    HANDLE  File;                   // File HANDLE != 0, если файл открыт
    HANDLE  FileMapping;            // Mapping File HANDLE != 0, если mapping создан
    LPVOID  Addr;                   // Addr != NULL, если mapview выполнен
    char    LastErrorMessage[512];  // сообщение об последней ошибке или 0x00
    time_t  lastsnaptime;           // дата последнего snap'a (time())
};
```

```
struct Element   // элемент
{
    Element();
    Element(const void* key, int keylength);                                                    // for Get
    Element(const void* key, int keylength, const void* payload, int  payloadlength);    // for Insert
    Element(Element* oldelement, const void* newpayload, int  newpayloadlength);         // for update
    const void*     key;                // значение ключа
    int             keylength;          // рахмер ключа
    const void*     payload;            // данные
    int             payloadlength;      // размер данных
};
```

```
HTHANDLE*  Create   //  создать HT
(
    int   Capacity,                   // емкость хранилища
    int   SecSnapshotInterval,        // периодичность сохранения в сек.
    int   MaxKeyLength,               // максимальный размер ключа
    int   MaxPayloadLength,           // максимальный размер данных
    const char  FileName[512]         // имя файла
);  // != NULL успешное завершение
```

```
HTHANDLE*   Open     //  открыть HT
(
    const char    FileName[512]         // имя файла
);  // != NULL успешное завершение
```

```
BOOL Snap         // выполнить Snapshot
(
    const HTHANDLE*    hthandle           // управление HT (File, FileMapping)
);
```

```
BOOL Insert      // добавить элемент в хранилище
(
    const HTHANDLE* hthandle,            // управление HT
    const Element*  element              // элемент
);  //  == TRUE успешное завершение
```

```
BOOL Delete      // удалить элемент в хранилище
(
    const HTHANDLE* hthandle,             // управление HT (ключ)
    const Element*  element               // элемент
);  //  == TRUE успешное завершение
```

```
Element* Get     //  читать элемент в хранилище
(
    const HTHANDLE* hthandle,              // управление HT
    const Element*  element                // элемент
);  //  != NULL успешное завершение
```

```
BOOL Update     //  именить элемент в хранилище
(
    const HTHANDLE* hthandle,            // управление HT
    const Element*  oldelement,          // старый элемент (ключ, размер ключа)
    const void*     newpayload,          // новые данные
    int             newpayloadlength     // размер новых данных
);  //  != NULL успешное завершение
```

```
char* GetLastError  // получить сообщение о последней ошибке
(
    HTHANDLE* ht                          // управление HT
);
```

```
    void print                            // распечатать элемент
    (
        const Element*  element           // элемент
    );


};
```

4. **Пример применения HT-API:** создание нового HT (Create), выполнение операций (Insert, Get, Delete, Update), синхронное сохранение (Snap), закрытие (Close), обработка ошибок (GetLastError).

```cpp
#include <Windows.h>
#include <iostream>
#include "HT.h"

int main()
{
    HT::HTHANDLE* ht = nullptr;
    try
    {
        ht = HT::Create(1000, 3, 10, 256, "D:\\HTspace.ht");    // создать HT-хранилище
        if (ht)  std::cout << "-- Create: success";
        else throw   " -- Create: error";

        if (HT::Insert(ht, new HT::Element("key222", 7, "payload", 8))) std::cout << "-- Insert:success";
        else throw   "-- insert:error";

        HT::Element* hte = HT::Get(ht, new HT::Element("key222", 7));
        if (hte)      std::cout << "Get: success";
        else throw   "-- Get: error";

        HT::print(hte);

        if (HT::Snap(ht)) std::cout << "Snap: success";
        else throw   "-- Snap: error";

        if (HT::Update(ht, hte, "newpayload", 11)) std::cout << "-- Update:success";
        else throw   "-- Update:error";

        HT::Element* hte1 = HT::Get(ht, new HT::Element("key222", 7));
        if (hte1)      std::cout << "Get: success";
        else throw   "-- Get: error";

        HT::print(hte1);

    if (HT::Delete(ht, hte1)) std::cout << "Delete: success";
    else  throw   "-- Delete: error";


    if (HT::Close(ht))  std::cout << "Close: success";
    else throw   "-- Close: error";

}
catch (char* msg)
{
    std::cout << msg << "\n";
    if (ht != nullptr)  std::cout << HT::GetLastError(ht);
}
```

5. **Пример применения HT-API:** открытие существующего HT (Open), выполнение операций, закрытие, обработка ошибок. (

```
HT::HTHANDLE* ht = nullptr;
try
{
    ht = HT::Open("D:\\HTspace.ht");    // открыть HT-хранилище
    if (ht)  std::cout << "-- Open: success";
    else throw   " -- Open: error";

    if (HT::Insert(ht, new HT::Element("key333", 7, "payload", 8))) std::cout << "-- Insert:success";
    else throw   "-- insert:error";

    HT::Element* hte = HT::Get(ht, new HT::Element("key222", 7));
    if (hte)      std::cout << "Get: success";
    else throw   "-- Get: error";


    if (HT::Close(ht))  std::cout << "Close: success";
    else throw   "-- Close: error";

}
catch (char* msg)
{
    std::cout << msg << "\n";
    if (ht != nullptr)  std::cout << HT::GetLastError(ht);
}
```

Задание 01.Windows

6. Разработайте приложение **OS10_01.**
7. Приложение **OS10_01** должно реализовывать **HT API.**
8. Приложение **OS10_01** должно демонстрировать выполнение всех функций **HT API.**

Задание 02.Windows

9. Разработайте статическую библиотеку **OS10_HTAPI.**
10. Разработайте приложение **OS10_02,** функционально-повторяющее **OS10_01,** но использующее библиотеку **OS10_HTAPI.**
11. Продемонстрируйте работоспособность приложения **OS10_02.**

Задание 03.Windows

12. Разработайте приложение **OS10_03,** применяющего библиотеку **OS10_HTAPI** и использующее одновременно 2 экземпляра HT-хранилища.
13. Продемонстрируйте работоспособность приложения **OS10_03.**

Задание 04.Windows

14. Разработайте **Test Case HT API.**
15. Разработайте тесты, соответствующие **Test Case HT API.**
16. Продемонстрируйте выполнение тестов.